

Разбор олимпиады

Анализ решений задач А-Е

Содержание

1	Задача А: Крестики-нолики	2
1.1	Условие	2
1.2	Входные данные	2
1.3	Выходные данные	2
1.4	Разбор	2
1.5	Код решения	3
2	Задача В: Параллелограмм	6
2.1	Условие	6
2.2	Входные данные	6
2.3	Выходные данные	6
2.4	Разбор	6
2.5	Код решения	7
3	Задача С: Парабола	9
3.1	Условие	9
3.2	Входные данные	9
3.3	Выходные данные	9
3.4	Примечание	9
3.5	Разбор	9
3.6	Код решения	10
4	Задача D: Вновь задача о черепашке	14
4.1	Условие	14
4.2	Входные данные	14
4.3	Выходные данные	14
4.4	Примечание	14
4.5	Разбор	14
4.5.1	Оптимальное решение	15
4.6	Код решения	15
5	Задача Е: Вновь настольная игра	19
5.1	Условие	19
5.2	Входные данные	19
5.3	Выходные данные	19
5.4	Примечание	19
5.5	Разбор	20
5.6	Код решения	21

1 Задача А: Крестики-нолики

1.1 Условие

Поликарп недавно узнал про игру в крестики-нолики на поле 3×3 . Он быстро освоил правила и обыграл всех своих друзей! Теперь Поликарп задумался над более сложными вопросами:

- Сколькими различными способами можно выиграть, играя за крестики?
- Сколькими различными способами можно выиграть, играя за нолики?
- Сколькими различными способами игра может закончиться вничью?

Помогите Поликарпу найти ответы на его вопросы!

Что считается игрой?

Пронумеруем клетки поля следующим образом:

$(0, 0)$, $(0, 1)$, $(0, 2)$ — верхняя строка

$(1, 0)$, $(1, 1)$, $(1, 2)$ — средняя строка

$(2, 0)$, $(2, 1)$, $(2, 2)$ — нижняя строка

Игра — это последовательность клеток, в которые по очереди делаются ходы. Первым всегда ходят крестики, затем нолики, и так далее по очереди.

Игра заканчивается, когда:

- один из игроков выстроил три своих символа в ряд (по горизонтали, вертикали или диагонали) — этот игрок выиграл;
- все 9 клеток заполнены, но никто не выстроил три в ряд — ничья.

Две игры считаются различными, если отличаются последовательностью ходов. Например, игры с последовательностями ходов $[(0,0), (1,1), (0,1), \dots]$ и $[(0,0), (1,1), (0,2), \dots]$ — это разные игры.

1.2 Входные данные

В этой задаче нет входных данных.

1.3 Выходные данные

Выведите единственную строку с тремя целыми числами, разделёнными пробелами: количество способов выиграть за крестики, количество способов выиграть за нолики, количество способов сыграть вничью.

1.4 Разбор

Эта задача решается полным перебором всех возможных игр с использованием рекурсии (DFS).

Ключевые идеи:

1. Представляем игровое поле как матрицу 3×3 , где 0 — пустая клетка, 1 — крестик, 2 — нолик
2. Используем рекурсивную функцию, которая на каждом шаге проверяет:

- Есть ли победитель (три в ряд)
 - Если поле заполнено — ничья
 - Иначе пробуем все возможные ходы текущего игрока
3. После каждого хода проверяем условия окончания игры
 4. Считаем количество различных исходов

Алгоритм:

1. Создаем функцию проверки победы для заданного игрока
2. Рекурсивная функция DFS:
 - Проверяем победу крестиков — увеличиваем счетчик
 - Проверяем победу ноликов — увеличиваем счетчик
 - Если поле заполнено и нет победителя — увеличиваем счетчик ничьих
 - Иначе для каждой пустой клетки делаем ход и вызываем DFS рекурсивно
3. Запускаем DFS с пустого поля, начиная с хода крестиков

Сложность: $O(9!)$ в худшем случае, но на практике значительно меньше из-за раннего завершения игр.

1.5 Код решения

```

1  #include <algorithm>
2  #include <cassert>
3  #include <chrono>
4  #include <iostream>
5  #include <map>
6  #include <numeric>
7  #include <ostream>
8  #include <random>
9  #include <ranges>
10 #include <set>
11 #include <stack>
12 #include <unordered_map>
13 #include <unordered_set>
14 #include <vector>
15
16 #define sz(a) ((int)((a).size()))
17 #define make_unique(x)
18
19     sort((x).begin(), (x).end());
20
21     (x).erase(unique((x).begin(), (x).end()), (x).end())
22
23 using namespace std;
24 mt1937 rnd(chrono::steady_clock::now().time_since_epoch().count());
25
26 typedef long long ll;
27 typedef long double ld;

```

```

26
27 bool check_for_win(auto &f, int who) {
28     for (int i = 0; i < 3; ++i) {
29         if (f[i][0] == who && f[i][1] == who && f[i][2] == who)
30             return true;
31         if (f[0][i] == who && f[1][i] == who && f[2][i] == who)
32             return true;
33     }
34     if (f[0][0] == who && f[1][1] == who && f[2][2] == who)
35         return true;
36     if (f[0][2] == who && f[1][1] == who && f[2][0] == who)
37         return true;
38
39     return false;
40 }
41
42 void dfs(auto &f, auto &x, auto &o, auto &d, int turn) {
43     if (check_for_win(f, 1)) {
44         ++x;
45         return;
46     } else if (check_for_win(f, 2)) {
47         ++o;
48         return;
49     } else {
50         int empty = 0;
51         for (int i = 0; i < 3; ++i) {
52             for (int j = 0; j < 3; ++j) {
53                 if (f[i][j] == 0)
54                     ++empty;
55             }
56         }
57         if (empty == 0) {
58             ++d;
59             return;
60         }
61         for (int i = 0; i < 3; ++i) {
62             for (int j = 0; j < 3; ++j) {
63                 if (f[i][j] == 0) {
64                     f[i][j] = turn;
65                     dfs(f, x, o, d, (turn == 1 ? 2 : 1));
66                     f[i][j] = 0;
67                 }
68             }
69         }
70     }
71 }
72
73 void solve() {
74     vector<vector<int>> field(3, vector<int>(3, 0));
75     int x_win = 0;
76     int o_win = 0;
77     int draw = 0;
78     dfs(field, x_win, o_win, draw, 1);

```

```
79     cout << x_win << ' ' << o_win << ' ' << draw << endl;
80 }
81
82 int32_t main() {
83     ios::sync_with_stdio(0);
84     cin.tie(0);
85     cout.tie(0);
86     int t = 1;
87     while (t--) {
88         solve();
89     }
90 }
```

2 Задача В: Параллелограмм

2.1 Условие

Поликарп недавно изучил на уроках геометрии параллелограммы и теперь ему надо решать домашнее задание на эту тему. Он решил все задачи, кроме одной.

На плоскости даны отрезок AB , точка C и треугольник XYZ .

Поликарпу нужно найти такие точки D и E , чтобы выполнялись следующие условия:

1. Точка E лежит на луче AC (то есть угол CAB равен углу EAB)
2. $ABDE$ является параллелограммом (AB параллельно ED , AE параллельно BD)
3. Площадь параллелограмма $ABDE$ равна площади треугольника XYZ

Помогите Поликарпу найти координаты точек D и E и решить последнюю задачу из его домашней работы.

2.2 Входные данные

Единственная строка содержит 12 вещественных чисел (с не более чем 3 знаками после десятичной точки), разделённых пробелом: $A_x A_y B_x B_y C_x C_y X_x X_y Y_x Y_z Z_x Z_y$. Точка A имеет координаты (A_x, A_y) , точка B — (B_x, B_y) , и так далее.

Гарантируется, что точки A , B и C не лежат на одной прямой. Гарантируется, что точки X , Y и Z не лежат на одной прямой. Гарантируется, что все числа находятся в диапазоне от -1000.0 до 1000.0 включительно.

2.3 Выходные данные

Выведите четыре вещественных числа, разделённые пробелом: $D_x D_y E_x E_y$ где точка D имеет координаты (D_x, D_y) , а точка E — (E_x, E_y) .

Все значения должны быть выведены с округлением до 3 знаков после десятичной точки.

Гарантируется, что все числа в ответе находятся в диапазоне от -1000000.0 до 1000000.0 включительно.

2.4 Разбор

Эта задача сводится к нахождению длины отрезка AE , при которой площадь параллелограмма $ABDE$ равна площади треугольника XYZ .

Ключевые идеи:

1. Вычисляем площадь треугольника XYZ через векторное произведение: $S = \frac{1}{2} |\vec{XY} \times \vec{XZ}|$
2. Площадь параллелограмма $ABDE$ равна $|\vec{AB}| \cdot h$, где h — высота, опущенная из E на AB
3. Высота $h = |\vec{AE}| \cdot \sin(\alpha)$, где α — угол между AB и AC
4. Длина AE : $|\vec{AE}| = \frac{S}{|\vec{AB}| \cdot \sin(\alpha)}$

Алгоритм:

1. Вычисляем площадь треугольника XYZ
2. Находим угол α между векторами AB и AC: $\cos(\alpha) = \frac{\overrightarrow{AB} \cdot \overrightarrow{AC}}{|\overrightarrow{AB}| \cdot |\overrightarrow{AC}|}$
3. Вычисляем требуемую длину AE
4. Находим точку E на луче AC: $E = A + \frac{|\overrightarrow{AE}|}{|\overrightarrow{AC}|} \cdot \overrightarrow{AC}$
5. Находим точку D: $D = B + \overrightarrow{AE}$ (по свойству параллелограмма)

2.5 Код решения

```

1  #include <algorithm>
2  #include <cassert>
3  #include <chrono>
4  #include <iomanip>
5  #include <iostream>
6  #include <map>
7  #include <numeric>
8  #include <ostream>
9  #include <random>
10 #include <ranges>
11 #include <set>
12 #include <stack>
13 #include <unordered_map>
14 #include <unordered_set>
15 #include <vector>
16
17 #define sz(a) ((int)((a).size()))
18 #define make_unique(x)
19
20     sort((x).begin(), (x).end());
21
22     (x).erase(unique((x).begin(), (x).end()), (x).end())
23
24 using namespace std;
25 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
26
27 typedef long long ll;
28 typedef long double ld;
29
30 template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
31
32 template <class T> struct Point {
33     typedef Point P;
34     T x, y;
35     explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
36     bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
37     bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
38     P operator+(P p) const { return P(x + p.x, y + p.y); }
39     P operator-(P p) const { return P(x - p.x, y - p.y); }
40     P operator*(T d) const { return P(x * d, y * d); }
41     P operator/(T d) const { return P(x / d, y / d); }

```

```

40 T dot(P p) const { return x * p.x + y * p.y; }
41 T cross(P p) const { return x * p.y - y * p.x; }
42 T cross(P a, P b) const { return (a - *this).cross(b - *this); }
43 T dist2() const { return x * x + y * y; }
44 double dist() const { return sqrt((double)dist2()); }
45 double angle() const { return atan2(y, x); }
46 P unit() const { return *this / dist(); }
47 P perp() const { return P(-y, x); }
48 P normal() const { return perp().unit(); }
49 P rotate(double a) const {
50     return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
51 }
52 friend ostream &operator<<(ostream &os, P p) {
53     return os << "(" << p.x << ", " << p.y << ")";
54 }
55 };
56
57 void solve() {
58     Point<double> a, b, c, x, y, z;
59     Point<double> zero{0.0, 0.0};
60     cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> x.x >> x.y >> y.x >>
61         y.y >>
62         z.x >> z.y;
63     if (a == zero && b == zero && c == zero && x == zero && y == zero &&
64         z == zero)
65         return;
66     auto xy = y - x;
67     auto xz = z - x;
68     double S = abs(xy.cross(xz) / 2.0);
69
70     auto ab = b - a;
71     auto ac = c - a;
72     double alpha = acos(ab.dot(ac) / (ab.dist() * ac.dist()));
73
74     double len = S / ab.dist() / sin(alpha);
75     auto ah = ac.unit() * len;
76     auto e = a + ah;
77     auto d = b + ah;
78     cout << setprecision(3) << fixed;
79     cout << d.x << ' ' << d.y << ' ' << e.x << ' ' << e.y << '\n';
80 }
81
82 int32_t main() {
83     ios::sync_with_stdio(0);
84     cin.tie(0);
85     cout.tie(0);
86     int t = 1;
87     while (t--) {
88         solve();
89     }
90 }

```

3 Задача С: Парабола

3.1 Условие

На очередном уроке алгебры Поликарпу рассказали про параболы. В этот раз ему нужна помощь со следующей задачей.

Дана парабола вида $y = x^2 + bx + c$ и три точки A, B, C . Точки A и C лежат на параболе, точка B лежит *внутри* параболы и ниже точек A, C ($A_x < B_x < C_x$, $A_y > B_y$, $C_y > B_y$). Из точки B проведены лучи BA и BC . Лучи отражаются от линии параболы по законам физики (угол падения равен углу отражения). Гарантируется, что каждый луч отразится от параболы не более одного раза перед тем, как пересечёт другой луч. Помогите Поликарпу найти точку пересечения лучей.

3.2 Входные данные

В первой строке через пробел даны два вещественных числа b и c .

Во второй строке через пробел даны шесть вещественных чисел $A_x, A_y, B_x, B_y, C_x, C_y$. Точка A имеет координаты (A_x, A_y) , точка B — (B_x, B_y) , точка C — (C_x, C_y) .

Гарантируется, что:

- Ответ для входных данных всегда существует.
- Все числа даны с точностью до пяти знаков после десятичной точки.
- Все числа находятся в диапазоне от -1000.0 до 1000.0 включительно.

3.3 Выходные данные

Выведите два вещественных числа, разделённые пробелом, - координаты x и y точки пересечения лучей.

Все значения должны быть выведены с округлением до 2 знаков после десятичной точки.

3.4 Примечание

В первом тесте оба луча отразятся от линии параболы по одному разу, после чего пересекутся в точке $(0, 2.65)$.

3.5 Разбор

Эта задача требует понимания отражения лучей от кривой поверхности.

Ключевые идеи:

1. Для отражения от параболы нужно найти нормаль в точке отражения
2. Касательная к параболе $y = x^2 + bx + c$ в точке (x_0, y_0) имеет направляющий вектор $(1, y'(x_0)) = (1, 2x_0 + b)$
3. Нормаль имеет направляющий вектор $(-y'(x_0), 1) = (-(2x_0 + b), 1)$
4. Угол отражения равен углу падения относительно нормали
5. Возможны два случая:

- Оба луча отражаются и затем пересекаются
- Один луч пересекается с отраженным вторым лучом до его отражения

Алгоритм:

1. Вычисляем производную параболы: $y'(x) = 2x + b$
2. Для каждой точки отражения (А и С) находим касательную $(1, y'(x_0))$ и нормаль $(-y'(x_0), 1)$
3. Вычисляем углы падения и отражения для лучей ВА и ВС относительно нормали
4. Строим отраженные лучи
5. Ищем точку пересечения различных комбинаций лучей:
 - Отраженный от А с отраженным от С
 - Отраженный от А с исходным ВС
 - Исходный ВА с отраженным от С
6. Выбираем первое найденное пересечение

3.6 Код решения

```

1  #include <cassert>
2  #include <chrono>
3  #include <iostream>
4  #include <map>
5  #include <numeric>
6  #include <ostream>
7  #include <iomanip>
8  #include <queue>
9  #include <random>
10 #include <ranges>
11 #include <set>
12 #include <stack>
13 #include <unordered_map>
14 #include <unordered_set>
15 #include <vector>
16
17 #define sz(a) ((int)((a).size()))
18 #define make_unique(x)
19
20     sort((x).begin(), (x).end());
21
22     (x).erase(unique((x).begin(), (x).end()), (x).end())
23
24 using namespace std;
25 mt1937 rnd(chrono::steady_clock::now().time_since_epoch().count());
26
27 typedef long long ll;
28 typedef long double ld;

```

```

28 const double EPS = 1E-9;
29
30 template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
31
32 template <class T> struct Point {
33     typedef Point P;
34     T x, y;
35     explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
36     bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
37     bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
38     P operator+(P p) const { return P(x + p.x, y + p.y); }
39     P operator-(P p) const { return P(x - p.x, y - p.y); }
40     P operator*(T d) const { return P(x * d, y * d); }
41     P operator/(T d) const { return P(x / d, y / d); }
42     T dot(P p) const { return x * p.x + y * p.y; }
43     T cross(P p) const { return x * p.y - y * p.x; }
44     T cross(P a, P b) const { return (a - *this).cross(b - *this); }
45     T dist2() const { return x * x + y * y; }
46     double dist() const { return sqrt((double)dist2()); }
47     double angle() const { return atan2(y, x); }
48     P unit() const { return *this / dist(); }
49     P perp() const { return P(-y, x); }
50     P normal() const { return perp().unit(); }
51     P rotate(double a) const {
52         return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
53     }
54     friend ostream &operator<<(ostream &os, P p) {
55         return os << "(" << p.x << ", " << p.y << ")";
56     }
57 };
58
59 template <class T> struct Line {
60     double a, b, c;
61
62     Line() {}
63     Line(Point<T> p, Point<T> q) {
64         a = p.y - q.y;
65         b = q.x - p.x;
66         c = -a * p.x - b * p.y;
67         norm();
68     }
69
70     void norm() {
71         double z = sqrt(a * a + b * b);
72         if (abs(z) > EPS)
73             a /= z, b /= z, c /= z;
74     }
75
76     double dist(Point<T> p) const { return a * p.x + b * p.y + c; }
77 };
78
79 #define det(a, b, c, d) (a * d - b * c)
80

```

```

81 inline bool betw(double l, double r, double x) {
82     return min(l, r) <= x + EPS && x <= max(l, r) + EPS;
83 }
84
85 template <class T>
86 bool intersect(Point<T> a, Point<T> b, Point<T> c, Point<T> d, Point<T>
    &ans) {
87     Line m(a, b);
88     Line n(c, d);
89     double zn = det(m.a, m.b, n.a, n.b);
90     ans.x = -det(m.c, m.b, n.c, n.b) / zn;
91     ans.y = -det(m.a, m.c, n.a, n.c) / zn;
92     return betw(a.x, b.x, ans.x) && betw(a.y, b.y, ans.y) &&
93         betw(c.x, d.x, ans.x) && betw(c.y, d.y, ans.y);
94 }
95
96 void solve() {
97     double b, c;
98     cin >> b >> c;
99     Point<double> A, B, C;
100    cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;
101
102    auto dy = [b](double x) -> double { return 2.0 * x + b; };
103    auto Anorm = Point(1.0, dy(A.x)).normal();
104    auto Cnorm = Point(1.0, dy(C.x)).normal();
105
106    auto Aangle = Anorm.angle() - (B - A).angle();
107    auto Cangle = (C - B).angle() - (Cnorm * -1.0).angle();
108
109    auto Anew = (B - A).rotate(Aangle + Aangle).unit() * 100.0;
110    auto Cnew = (B - C).rotate(-(Cangle + Cangle)).unit() * 100.0;
111
112    cout << setprecision(2) << fixed;
113    Point<double> ans;
114    if (intersect(A, A + Anew, C, C + Cnew, ans)) {
115        cout << ans.x << ' ' << ans.y << endl;
116    } else if (intersect(A, A + Anew, B, C, ans)) {
117        cout << ans.x << ' ' << ans.y << endl;
118    } else if (intersect(B, A, C, C + Cnew, ans)) {
119        cout << ans.x << ' ' << ans.y << endl;
120    } else {
121        assert(false);
122    }
123 }
124
125 int32_t main() {
126     ios::sync_with_stdio(0);
127     cin.tie(0);
128     cout.tie(0);
129     int t = 1;
130     while (t--) {
131         solve();
132     }

```


4 Задача D: Вновь задача о черепашке

4.1 Условие

Дано квадратное поле $N \times N$, в каждой клетке есть некоторое число монет. Черепашка начинает с левого нижнего (с координатами $(1, 1)$) угла и хочет оказаться в правом верхнем углу (с координатами (N, N)). Черепашка за время Вашей учебы выросла, теперь за 1 ход она может переместиться до 1 строки по горизонтали и до 8 строк по вертикали: из клетки (x, y) она может перейти в $(x, y+1)$, $(x, y+2)$, ..., $(x, y+8)$, $(x+1, y)$, $(x+1, y+1)$, ..., $(x+1, y+8)$. она подбирает все монетки из посещенных клеток. Необходимо добраться до правого верхнего угла с максимальным числом монет, а также восстановить путь, которым эта сумма монет была получена. Если ответов несколько, можно вывести любой.

Обратите внимание на ограничение по памяти.

Эту задачу можно решить только на c++

4.2 Входные данные

В первой строке через пробел вводится 4 числа N , A , B и M , ($1 < N \leq 10000$), ($1 \leq A, B, M \leq 10^9 + 7$) - размеры поля и параметры соответственно. Во второй строке через пробел вводится N чисел a_i ($1 \leq a_i \leq 1e9 + 7$). В третьей строке через пробел вводится N чисел b_j ($1 \leq b_j \leq 1e9 + 7$). В клетке с координатами (i, j) находится $((A * a_i + B * b_j) \bmod M - 100)$ монет.

4.3 Выходные данные

В первой строке выведите одно число T - количество клеток в пути. В следующих T строках выведите через пробел 2 числа - координаты очередной клетки восстановленного пути, первая из которых $(1, 1)$, последняя - (N, N)

4.4 Примечание

Жюри участников любит, поэтому после семплов добавлены по 2 теста с $N=(1000; 2000; 3000; 4000; 5000; 5500; 6000; 6500; 7000; 7500; 8000; 8500; 9000; 9500)$, чтобы Вы могли понять сколько памяти ест Ваше решение. Тесты 29-30 имеют $N=9500$, тесты 19-20 имеют $N=7000$, тесты 11-12 имеют $N=5000$.

Схема поля (ось Y направлена вверх, X — вправо):

$Y \setminus X$	1 ($a = 10$)	2 ($a = 20$)	3 ($a = 30$)
3 ($b = 3$)	$(10 + 3) - 100 = -\mathbf{87}$	$(20 + 3) - 100 = -\mathbf{77}$	$(30 + 3) - 100 = -\mathbf{67}$
2 ($b = 2$)	$(10 + 2) - 100 = -\mathbf{88}$	$(20 + 2) - 100 = -\mathbf{78}$	$(30 + 2) - 100 = -\mathbf{68}$
1 ($b = 1$)	$(10 + 1) - 100 = -\mathbf{89}$	$(20 + 1) - 100 = -\mathbf{79}$	$(30 + 1) - 100 = -\mathbf{69}$

Итоговый баланс: $-89 + (-77) + (-67) = -233$.

4.5 Разбор

Эта задача является классической задачей динамического программирования с ограничением по памяти.

Стандартное решение потребует $O(N^2)$ памяти для хранения DP и восстановления пути, что превышает лимит при $N = 10000$.

Ключевые идеи авторского решения:

1. Разбиваем поле на квадратные блоки размером $C \times C$
2. За один запуск восстанавливаем путь только по одному блоку
3. Запускаем алгоритм $\frac{2N}{C}$ раз для восстановления всего пути
4. Используем кодирование ходов: каждый ход (dx, dy) кодируется как $dx \times 9 + dy$

Алгоритм:

1. Выбираем размер блока $C = 2000$ (компромисс между временем и памятью)
2. Для текущего блока:
 - Вычисляем DP для достижения финала этого блока
 - Сохраняем информацию о предыдущих ходах только для этого блока
 - Восстанавливаем путь по блоку
3. Переходим к следующему блоку и повторяем
4. Объединяем восстановленные пути всех блоков

Сложность:

- Время: $O(\frac{2N^3}{C})$
- Память: $O(N + C^2)$

4.5.1 Оптимальное решение

Задачу можно решить оптимальнее. Например, для каждой клетки запоминать не прошлую, а самую нижнюю из тех, что была в столбце $X_{mid} = \frac{(X_{start} + X_{finish})}{2}$, пусть это MID_Y.

После этого рекурсивно можно решать задачу

$(X_{start}, X_{finish}, X_{mid}, Y_{mid})$ и $(X_{mid}, Y_{mid}, X_{finish}, Y_{finish})$.

Можно доказать, что это решение имеет сложность $O(N^2)$ по времени и $O(N)$ по памяти.

4.6 Код решения

```

1  #include <cassert>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
7  const int MAXN = 10000, kBLOCK_SIZE = 2000;
8  long long kN, kA, kB, kM;
9  int arrA[MAXN], arrB[MAXN];
10 // Let's code each move (dx, dy) as (dx * 9 + dy) to get a number from
    1 to 17.
11 // Each move - one char.
12 char reversedPath[MAXN + MAXN];
13 int reversedPathSize = 0;
    
```

```

14 // contains same codes as reversedPath
15 char dpFrom[kBLOCK_SIZE][kBLOCK_SIZE];
16 long long dpPrevColumn[MAXN];
17 long long dpCurrentColumn[MAXN];
18
19 int GetCoins(int x, int y) { return (kA * arrA[x] + kB * arrB[y]) % kM
    - 100; }
20
21 int RestoreBlock(int currentFinish) {
22     int xMax = currentFinish / kN;
23     int yMax = currentFinish % kN;
24     for (int currentX = 0; currentX <= xMax; ++currentX) {
25         for (int currentY = 0; currentY <= yMax; ++currentY) {
26             bool somethingFound = false;
27             char bestDx = 0;
28             char bestDy = 0;
29             if (currentX != 0) {
30                 for (int prevY = currentY - 8; prevY <= currentY; ++prevY) {
31                     if (prevY >= 0) {
32                         if (!somethingFound ||
33                             bestDx == 0 &&
34                             dpCurrentColumn[currentY - bestDy] < dpPrevColumn[
35                                 prevY] ||
36                             bestDx == 1 &&
37                             dpPrevColumn[currentY - bestDy] < dpPrevColumn[
38                                 prevY]) {
39                             somethingFound = true;
40                             bestDx = 1;
41                             bestDy = currentY - prevY;
42                         }
43                     }
44                 }
45             }
46             for (int prevY = currentY - 8; prevY < currentY; ++prevY) {
47                 if (prevY >= 0) {
48                     if (!somethingFound ||
49                         bestDx == 0 &&
50                         dpCurrentColumn[currentY - bestDy] < dpCurrentColumn[
51                             prevY] ||
52                         bestDx == 1 &&
53                         dpPrevColumn[currentY - bestDy] < dpCurrentColumn[
54                             prevY]) {
55                         somethingFound = true;
56                         bestDx = 0;
57                         bestDy = currentY - prevY;
58                     }
59                 }
60             }
61             if (!somethingFound) {
62                 dpCurrentColumn[currentY] = GetCoins(currentX, currentY);
63             } else {
64                 if (bestDx == 0) {
65                     dpCurrentColumn[currentY] =

```

```

62         dpCurrentColumn[currentY - bestDy] + GetCoins(currentX,
63             currentY);
64     } else {
65         dpCurrentColumn[currentY] =
66             dpPrevColumn[currentY - bestDy] + GetCoins(currentX,
67                 currentY);
68     }
69     if (xMax - currentX < kBLOCK_SIZE && yMax - currentY <
70         kBLOCK_SIZE) {
71         dpFrom[xMax - currentX][yMax - currentY] = bestDx * 9 +
72             bestDy;
73     }
74     swap(dpCurrentColumn, dpPrevColumn);
75 }
76 int currentX = xMax;
77 int currentY = yMax;
78 while (xMax - currentX < kBLOCK_SIZE && yMax - currentY < kBLOCK_SIZE
79     &&
80     !(currentX == 0 && currentY == 0)) {
81     reversedPath[reversedPathSize++] = dpFrom[xMax - currentX][yMax -
82         currentY];
83     currentX -= reversedPath[reversedPathSize - 1] / 9;
84     currentY -= reversedPath[reversedPathSize - 1] % 9;
85 }
86 return currentX * kN + currentY;
87 }
88
89 int main() {
90     std::cin.tie(nullptr);
91     std::ios_base::sync_with_stdio(false);
92     cin >> kN >> kA >> kB >> kM;
93     for (int i = 0; i < kN; ++i) {
94         cin >> arrA[i];
95     }
96     for (int i = 0; i < kN; ++i) {
97         cin >> arrB[i];
98     }
99     // In this program positions is (0, 0), ... (N-1, N-1)
100    // Let's code our position (x, y) as int x*N+y
101    int currentFinish = kN * kN - 1; // (N-1)*N+(N-1) = (N-1)(N+1) = N
102        ^2-1
103    while (currentFinish) {
104        currentFinish = RestoreBlock(currentFinish);
105    }
106    cout << reversedPathSize + 1 << "\n1 1\n";
107    int currentX = 1, currentY = 1;
108    for (int i = reversedPathSize - 1; i >= 0; --i) {
109        currentX += reversedPath[i] / 9;
110        currentY += reversedPath[i] % 9;
111        cout << currentX << ' ' << currentY << '\n';
112    }

```

```
108     assert(currentX == kN);  
109     assert(currentY == kN);  
110     return 0;  
111 }
```

5 Задача Е: Вновь настольная игра

5.1 Условие

Каждый в детстве играл в ходилку: есть некоторая трасса, участники по очереди бросают кубик, делают столько шагов вперед, сколько выпало. Побеждает тот, кто первый дойдет до финиша.

Чуть более формально, трассу можно представить в виде графа из N вершин, где из вершины i есть ребро в вершину $i + 1$, кроме вершины N , которая является финишем. Вершина 1 является стартовой. Игроки делают ход по очереди, на кубике равновероятно выпадает 1, 2, 3, 4, 5, 6. Если выпало число X , а фишка игрока стоит в вершине i , то **стартовой** вершиной называется i , **конечной** вершиной называется $i + X$, а **промежуточными** называются $i + 1, i + 2, \dots, i + X - 1$. Если игроку осталось 3 ребра до финиша, то для победы достаточно выпадения 3, 4, 5, 6.

Чтобы игра была интереснее, на карте встречаются стрелки, переносящие игрока на несколько вершин вперед или назад. Каждая стрелка начинается в вершине t_1 и заканчивается в t_2 . Перемещение по стрелке из t_1 в t_2 происходит только тогда, когда t_1 стала **конечной** вершиной хода.

Итого каждый ход игрока имеет вид:

1. Подбрасывание кубика, перемещение фишки в соответствии с выпавшим числом очков
2. Возможное перемещение по стрелке, если такая имеется в **конечной** вершине. Если переход по стрелке был, а из ее конца исходит очередная стрелка, то второй переход по стрелке **не происходит**

Если после пункта 2 фишка игрока оказалась в вершине N , считаем что игрок дошел до финиша. Оцените среднее количество ходов, которое потребуется для прохождения данной трассы.

5.2 Входные данные

В первой строке через пробел даны 2 числа N и M ($2 \leq N \leq 800, 0 \leq M \leq N - 2$) - число вершин на поле и число стрелок соответственно.

В следующих M строках через пробел даны 2 числа t_1 и t_2 , ($1 < t_1 < N, 1 \leq t_2 \leq N$) - начала и концы стрелок.

Гарантируется, что все t_1 различны.

Гарантируется, что игра проходимая и ответ не более чем 10^6

5.3 Выходные данные

Выведите одно вещественное число с тремя знаками после запятой - сколько в среднем ходов требуется для прохождения игры.

5.4 Примечание

В этой задаче баллы выставляются за группы тестов. Каждая группа будет тестироваться только при успешном тестировании предыдущей.

Группа	Баллы	Ограничения
1	9	$N \leq 6$
2	9	$N \leq 12$
3	72	$N \leq 90$
4	10	$N \leq 800$

Пояснение к первому тесту из условия: Обозначим за x_i количество ходов, необходимое для достижения финиша из вершины i , тогда

$$x_5 = 0$$

$$x_2 = \frac{1}{6}x_1 + \frac{1}{6}x_2 + \frac{4}{6}x_5 + 1 \text{ (не было бы стрелок, было бы } \frac{1}{6}x_3 + \frac{1}{6}x_4 + \frac{4}{6}x_5 + 1)$$

$$x_1 = \frac{1}{6}x_2 + \frac{1}{6}x_1 + \frac{1}{6}x_2 + \frac{3}{6}x_5 + 1$$

Из первого получим $x_2 = \frac{1}{5}x_1 + \frac{6}{5}$, подставим во второе $x_1(1 - \frac{1}{6} - \frac{1}{3}\frac{1}{5}) = \frac{1}{3}\frac{6}{5} + 1$, откуда
 ВЫХОДИТ $x_1 = \frac{30}{30-5-2}\frac{7}{5} = \frac{42}{23} = 1,826$

5.5 Разбор

С наличием стрелок назад значение DP может выражаться через еще необработанные клетки или через себя же. Давайте эту проблему решим.

Для этого будем двигаться не с конца в начало, а наоборот - с начала в конец.

Первая клетка выражается через клетки правее или через себя. Например, из клетки 4 ведет стрелка в 1, а из 2, 3, 5, 6, 7 стрелок нет. Тогда можно сказать что $x_1 = \frac{x_2+x_3+x_1+x_5+x_6+x_7}{6} + 1$. Или же $x_1 = \frac{x_2+x_3+x_5+x_6+x_7}{5} + \frac{6}{5}$

То есть мы получили x_1 выраженную через x с большим номером.

Перейдем к x_2 : оно выражается через x большие нас, возможно x_2 и, возможно x_1 . Давайте избавимся от x_1 через выражение с прошлого шага. Аналогично прошлому шагу получим x_2 , выраженное через x с большим номером.

Давайте поддерживать инвариант, что каждый x выражается только через еще необработанные x . Для этого, если формула x_1 содержит x_2 , подставим в него x_2 выраженное через x с номером 3 и больше.

Алгоритм решения:

1. Для каждого x сохраним $N + 1$ коэффициент, описывающий выражение для получения x . $(N + 1)$ -й коэффициент — это изначально единица, которая в примере для x_1 стала числом $\frac{6}{5}$
2. Получаем выражение для очередного x_i . Мы знаем что инвариант соблюдается, поэтому после упрощения выражения ненулевыми будут коэффициенты с номерами i и больше, поскольку все обработанные x выражены через них
3. После выражения x_i через $i + 1, i + 2, \dots$ проходимся по $x_1, x_2, \dots, x_{i-1}$ и для каждого выражения "прибавляем строчку x_i , умноженную на коэффициент перед x_i ". Например $x_1 = x_4 + \frac{1}{2}x_3$, а $x_3 = \frac{1}{3}x_4 + 1$, тогда, если бы $i = 3$, то $\text{coeffs}[x_1][x_4] += \text{coeffs}[x_1][x_3] \times \text{coeffs}[x_3][x_4]$, $\text{coeffs}[x_1][x_{N+1}] += \text{coeffs}[x_1][x_3] \times \text{coeffs}[x_3][x_{N+1}]$, после прибавления делаем присвоение $\text{coeffs}[x_1][x_3] = 0$
4. Как i стало равным N — задача решена. Инвариант соблюдался, значит каждый x выражен через x_N и x_{N+1} . x_N очевидно 0, так как мы уже на финише, а коэффициент x_{N+1} — свободный член. Фактически, мы получили что $x_i = \text{coeffs}[x_i][x_{N+1}]$, нам надо вывести x_1 , то есть ответ лежит в $\text{coeffs}[x_1][x_{N+1}]$

5.6 Код решения

```

1  #include <cassert>
2  #include <iomanip>
3  #include <iostream>
4  #include <vector>
5
6  using namespace std;
7
8  int main() {
9      std::cin.tie(nullptr);
10     std::ios_base::sync_with_stdio(false);
11     int n, m;
12     std::cin >> n >> m;
13     vector<int> resolve(n);
14     for (int i = 0; i < n; ++i) {
15         resolve[i] = i;
16     }
17     for (int i = 0; i < m; ++i) {
18         int t1, t2;
19         std::cin >> t1 >> t2;
20         --t1;
21         --t2;
22         resolve[t1] = t2;
23     }
24     double coef = (double)1.0 / 6.0;
25     // node n is one.
26     // invariant: each index in [0, node] is resolved with [node+1, n +
27     // 1]
28     std::vector<std::vector<double>> resolve_coeffs(n,
29                                                     std::vector<double>(n
30                                                         + 1));
31     for (int node = 0; node < n - 1; ++node) {
32         for (int score = 1; score <= 6; ++score) {
33             int finish = resolve[min(n - 1, node + score)];
34             if (finish >= node) {
35                 resolve_coeffs[node][finish] += coef;
36             } else {
37                 // get rid of indexes less than node
38                 for (int i = 0; i <= n; ++i) {
39                     resolve_coeffs[node][i] += coef * resolve_coeffs[finish][i];
40                 }
41             }
42             // do not forget step moving us to finish
43             resolve_coeffs[node][n] += 1;
44             // get rid of node in equation getting node
45             for (int i = 0; i <= n; ++i) {
46                 if (i != node) {
47                     resolve_coeffs[node][i] /= (1 - resolve_coeffs[node][node]);
48                 }
49             }
50             resolve_coeffs[node][node] = 0;

```

```
50
51 // and we should get rid of node coefficients in indexes [0, node]
    to keep
52 // invariant
53 for (int index = 0; index < node; ++index) {
54     for (int i = index + 1; i <= n; ++i) {
55         resolve_coeffs[index][i] +=
56             resolve_coeffs[index][node] * resolve_coeffs[node][i];
57     }
58     resolve_coeffs[index][node] = 0;
59 }
60 }
61 // now each node is resolved with [n - 1] which is 0 and [n] which is
    1.
62 // so, answer for each node X is resolve_coeffs[X][n - 1] * 0 +
63 // resolve_coeffs[X][n] * 1 = resolve_coeffs[X][n]
64 std::cout << setprecision(3) << fixed << resolve_coeffs[0][n] << std
    ::endl;
65 assert(resolve_coeffs[0][n] <= 1e6);
66 return 0;
67 }
```